

全齡向程式學習與運算思維培養的平台開發

**Development of a Programming Learning
and Computational Thinking Platform for
All Ages**

學生：劉翁碩、林芷如

指導教授：黃悅民

第一章 摘要

在參考了均一教育平台、Blockly Games 等強調運算思維或是程式教育的平台，我觀察到大多數平台會提供遊戲化教學，以降低學習門檻。但大多數平台僅專注在「運算思維教學遊戲化」或「程式編寫」其中一項，較少兩者兼顧。

為了與現有平台做出差異化，我設計了結合了**運算思維培養的小遊戲、程式碼觀念測驗和實際編寫環境**，以及透過**AI 輔助學習**的學習平台。此平台使用**Flask** 框架，搭配**HTML**、**CSS**、**JavaScript** 實作前端互動介面，並採用**SQLite** 資料庫儲存並管理使用者資料、教學影片、測驗結果紀錄等，並整合多項**API**（**Cohere**、**Judge0**、**QuizAPI**、**YouTube Data API** 等），規劃完整一套「學習 → 觀念測驗 → 實際操作 → 檢討並追蹤」的流程。

此平台具備以下特色：

- 融合劇情的**任務式遊戲**讓使用者加深了解運算思維四大要點。
- 以**圖像式程式編輯工具 Blockly**，讓學習者能透過視覺化的方式學習程式語言的條件判斷、順序執行、迴圈等邏輯。
- 提供教師上傳教學影片，並結合**QuizAPI** 出題。
- 系統運用**AI** 以及**Judge0 API** 為學生提供題目解析以及程式批改的建議輔助學習成果的評估。
- 整合**Google Custom Search API** 作為**爬蟲**工具，自動搜尋相關主題文章或教學內容，作為補強與延伸閱讀。
- 提供錯題回顧系統，並以**Chart.js** 視覺化學生的學習成果。
- 透過**APScheduler** 工作流程管理，每日定時寄送學習成果報告與錯題整理至學生 Gmail。

第二章 系統設計

2.1 系統架構

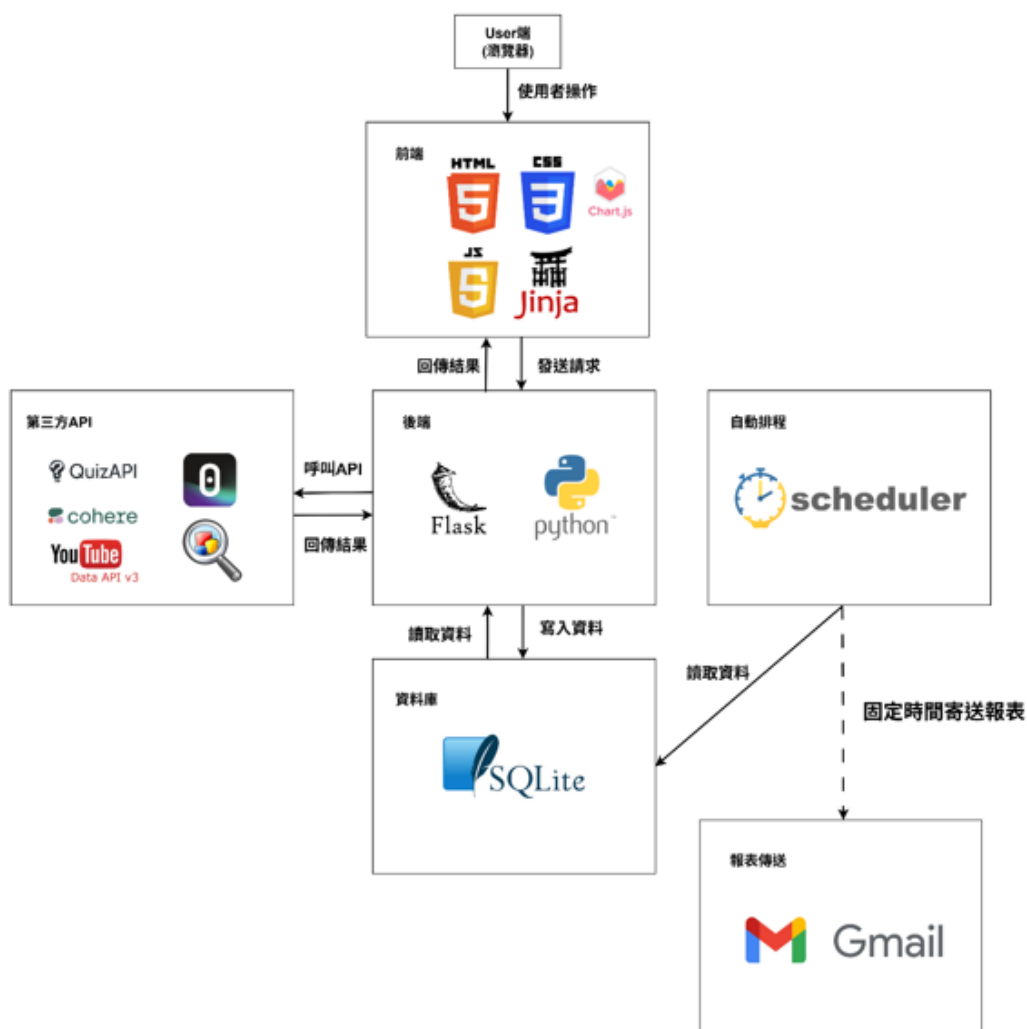


圖 2.1 網頁系統架構圖

2.2 前端設計

本網頁採用 **Flat Design**（扁平化設計）視覺風格，圖示搭配簡要的文字，讓使用者能快速上手，確保使用者能快速找到自己需要的功能。且按鈕皆設有

即時回饋，幫助使用者理解目前的操作狀態。

網頁前端使用 HTML、CSS、JavaScript 建構，頁面主要分為三大區域：

- 導覽列：顯示登入使用者，以及提供收起側邊欄和在淺色/深色模式中切換的按鈕。
- 側邊欄：列出所有主要功能頁面，使用者可快速切換至不同的區域。
- 主內容區：透過 **Jinja2** 模板載入不同頁面的專屬內容，方便依需求調整版面。

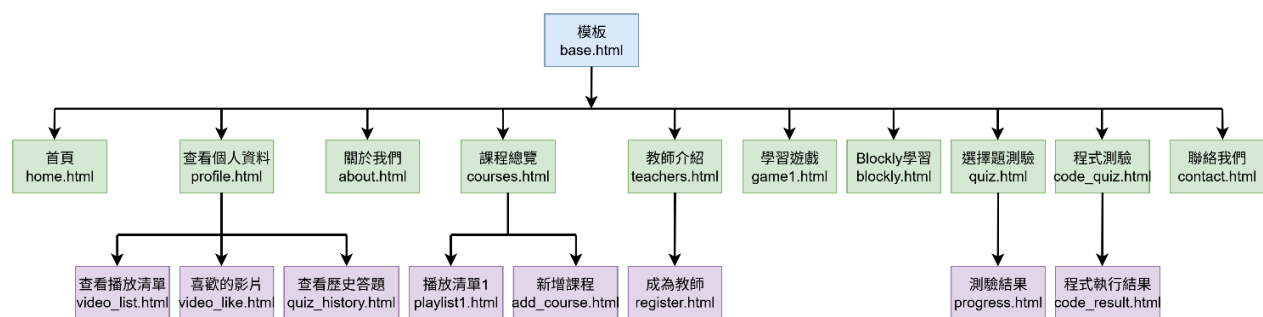


圖 2-2 前端頁面結構圖



圖 2-3 繼承 base.html 模板的 home.html(首頁)

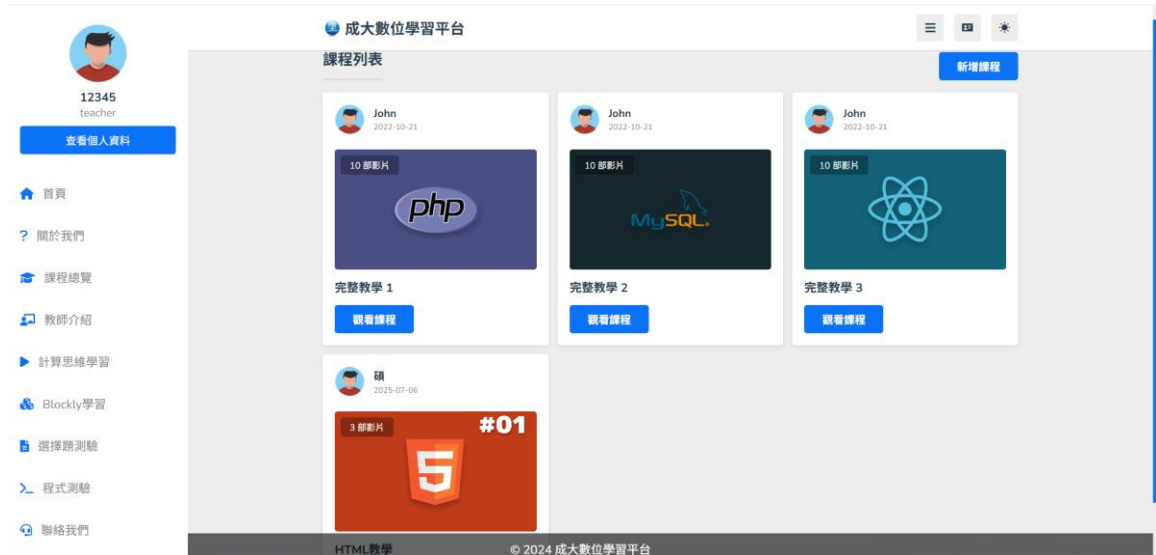


圖 2-4 課程總覽頁面

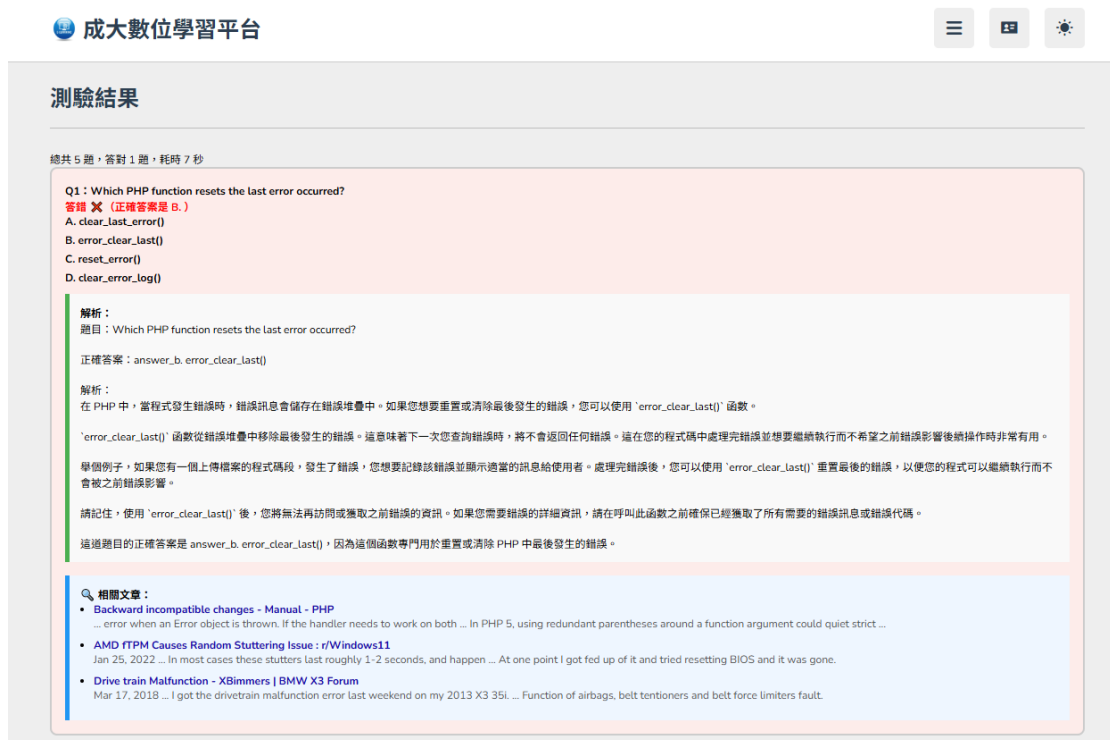


圖 2-4 題目解析頁面

2.3 後端設計

本網頁採用基於 Python 的 Flask 應用框架，此框架具輕量化特色。並結合多項模組及 API 整合，例如提供使用者管理、課程管理、測驗系統、AI 助教等模組。

- 路由設計：透過 Flask 的路由機制定義前端和後端的溝通街口，每個路由都對應到一個 URL 以處理不同的操作

例如：`@app.route("/login", method=['GET', 'POST'])`，Flask 路由可支援多種 HTTP 的方法(GET 和 POST)，根據不同請求執行相對應的邏輯處理。

- 使用者與權限管理：提供註冊、登入功能，並用 Flask Session 管理登入狀態。且根據角色(老師或是學生)控制不同頁面的使用權限，如只有老師能進入上傳課程的頁面。
- 課程管理：老師可新增課程以及影片並藉由後端傳送至資料庫。
- 測驗系統管理：整合 QuizAPI 及 Judge0 API，並藉由後端將 API 處理好的資料儲存至資料庫或在前端顯示。
- 延伸文章爬蟲功能：後端整合 Google Custom Search API，根據學生的題目以及作答搜尋相關教學資源。
- AI 助教：後端作為中介，將使用者的請求傳送到 Cohere API，提供題目的解析以及程式碼撰寫的修改建議。
- 資料儲存：使用 Flask-SQLAlchemy 結合 SQLite 儲存所有資料。

2.4 遊戲設計

本節說明此平台所設計的互動式教學遊戲，遊戲分為兩個類別，一為任務式的運算思維培養導向遊戲，其二為 Blockly 程式視覺化遊戲。透過遊玩遊戲，幫助使用者在遊玩中培養計算思維四大要點以及學習程式語言必備的邏輯能力。

2.4.1 任務式遊戲

透過劇情化的遊戲，讓學生在解決任務的同時可以沉浸在劇情當中，幫助增加學習動機。同時培養出計算思維四大要點的能力：

- (1) 拆解問題：將一個大問題拆解成多個子問題。在任務 1 當中，使用者必須將飛船維修拆解成四個子問題，並排列出正確順序。
- (2) 規律辨識：從複雜的問題當中，辨識出反覆出現的特徵。在任務 2 當中，使用者必須從一串有規律的燈泡當中，識別下一個燈泡應為亮還是暗。
- (3) 抽象化：只專注問題當中的核心，忽略細節以加強理解。在任務 3 當中，使用者須抽象化物體的類別，忽略顏色、大小等不必要的特徵。
- (4) 演算法設計：將思考過程具體化並設計一套具邏輯的解決問題順序。在任務 4 當中，使用者須建立一套流程，幫助飛船逃離迷宮。

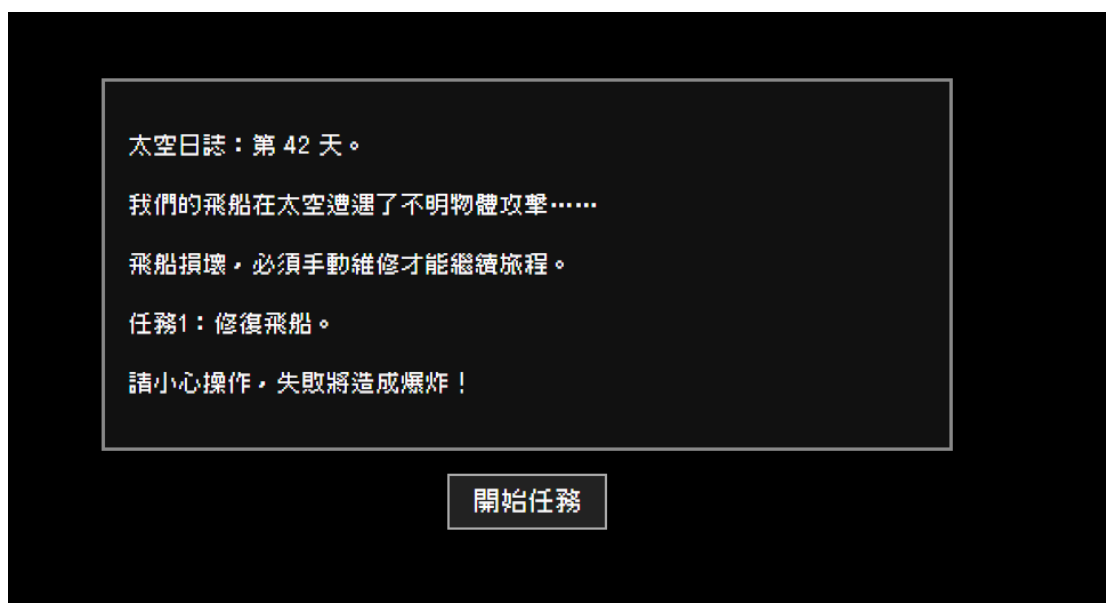


圖 2-5 遊戲前導劇情設計



圖 2-6 任務 1：運算思維第一步驟 - 拆解問題



圖 2-7 任務 2：運算思維第二步驟 - 規律辨識



圖 2-8 任務 3：運算思維第三步驟—抽象化



圖 2-9 任務 4：運算思維第四步驟－演算法設計

```

1 <div id="level2" style="display:none; text-align:center; padding:50px;">
2   <h2>訊號辨識--規律辨識</h2>
3   <p>觀察下列燈泡的閃爍順序(由左至右)，判斷其規律。</p>
4   <div id="light-sequence" style="margin-top: 20px;">
5     
6     
7     
8     
9     
10    
11    
12    
13  </div>
14
15  <button class="next-btn" id="analyze-btn" style="margin-top: 20px;">我看出規律了</button>
16
17  <div id="choice-buttons" style="margin-top: 20px; display:none;">
18    <p>請選擇下一個燈泡的狀態：</p>
19    <button class="next-btn" data-answer="on">亮</button>
20    <button class="next-btn" data-answer="off">暗</button>
21  </div>
22
23  <p id="pattern-feedback" style="margin-top: 20px;"></p>
24 </div>
25

```

圖 2-10 任務 2 之前端設計

```

function showSecondLevel() {
  const level2 = document.getElementById("level2");
  level2.style.display = "block";

  const analyzeBtn = document.getElementById("analyze-btn");
  const choiceButtons = document.getElementById("choice-buttons");
  const feedback = document.getElementById("pattern-feedback");

  const correctNext = "off";

  analyzeBtn.addEventListener("click", () => {
    analyzeBtn.style.display = "none";
    choiceButtons.style.display = "block";
  });

  choiceButtons.querySelectorAll("button").forEach(btn => {
    btn.addEventListener("click", () => {
      const answer = btn.getAttribute("data-answer");
      if (answer === correctNext) {
        feedback.textContent = "答對了！你找到正確規律！";
        feedback.style.color = "lightgreen";
        setTimeout(() => {
          showTransitionToLevel3();
        }, 1500);
      } else {
        feedback.textContent = "答錯了，再觀察一下燈泡的變化！";
        feedback.style.color = "red";
      }

      choiceButtons.querySelectorAll("button").forEach(b => b.disabled = true);
    });
  });
}

```

圖 2-11 控制任務 2 的 JavaScript 函式

2.4.2 Blockly 程式視覺化遊戲

此設計為一個以籃球為主題的 Blockly 小遊戲，使用者可透過拖曳積木指令控制球場上的角色。在遊戲設計上，AI 對手會自動追蹤玩家位置並靠近，若對手距離過近，會導致我方球員投籃失敗，因此使用者必須控制角色閃避對手的防阻並嘗試投籃。透過此一遊戲，希望培養使用者對於運算思維的能力及程式碼的邏輯運作。

此遊戲同時整合 Cohere API，能對目前畫面中的積木進行解釋，幫助使用者理解目前整體程式的運作邏輯，進一步的調整積木順序。

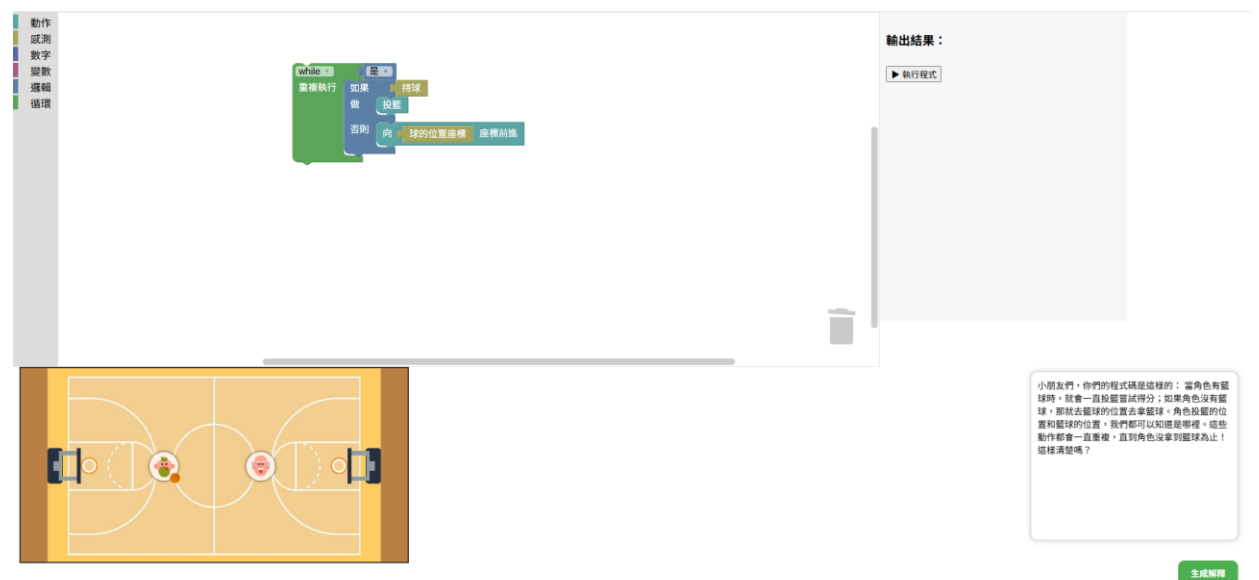


圖 2-12 Blockly 籃球遊戲

```

Blockly.JavaScript['move_angle'] = function(block) {
  const angle = Blockly.JavaScript.valueToCode(block, 'ANGLE', Blockly.JavaScript.ORDER_ATOMIC) || '0';
  const duration = Blockly.JavaScript.valueToCode(block, 'DURATION', Blockly.JavaScript.ORDER_ATOMIC) || '1';

  let nextCode = '';
  if (block.nextConnection && block.nextConnection.targetBlock()) {
    nextCode = Blockly.JavaScript.blockToCode(block.nextConnection.targetBlock());
  }

  return `moveAngle(${angle}, ${duration}, function() {\n${nextCode}});\n`;
};

```

圖 2-13 將積木轉為 JavaScript 程式碼以操控球員

```

function moveAIToPlayer() {
  const dx = playerX - aiX;
  const dy = playerY - aiY;
  const distance = Math.sqrt(dx * dx + dy * dy);
  if (distance < 2) return;

  const speed = 1.5;
  const vx = dx / distance;
  const vy = dy / distance;
  aiX += vx * speed;
  aiY += vy * speed;

  updateAIPosition();

  if (isPlayerTouchingAI()) {
    stopMove();
    showGameOver();
  }
}

```

圖 2-14 AI 對手自動靠近球員的程式碼片段

2.5 資料庫設計

本系統採用 SQLite 作為資料庫，並搭配 Flask-SQLAlchemy 作為 ORM(物件關聯對應)工具，將資料表抽象為 Python 類別，進行資料表的建立以及相關操作，並同時可定義資料表之間的關聯，輔以 Flask-Migrate 管理資料庫的版本和遷移。

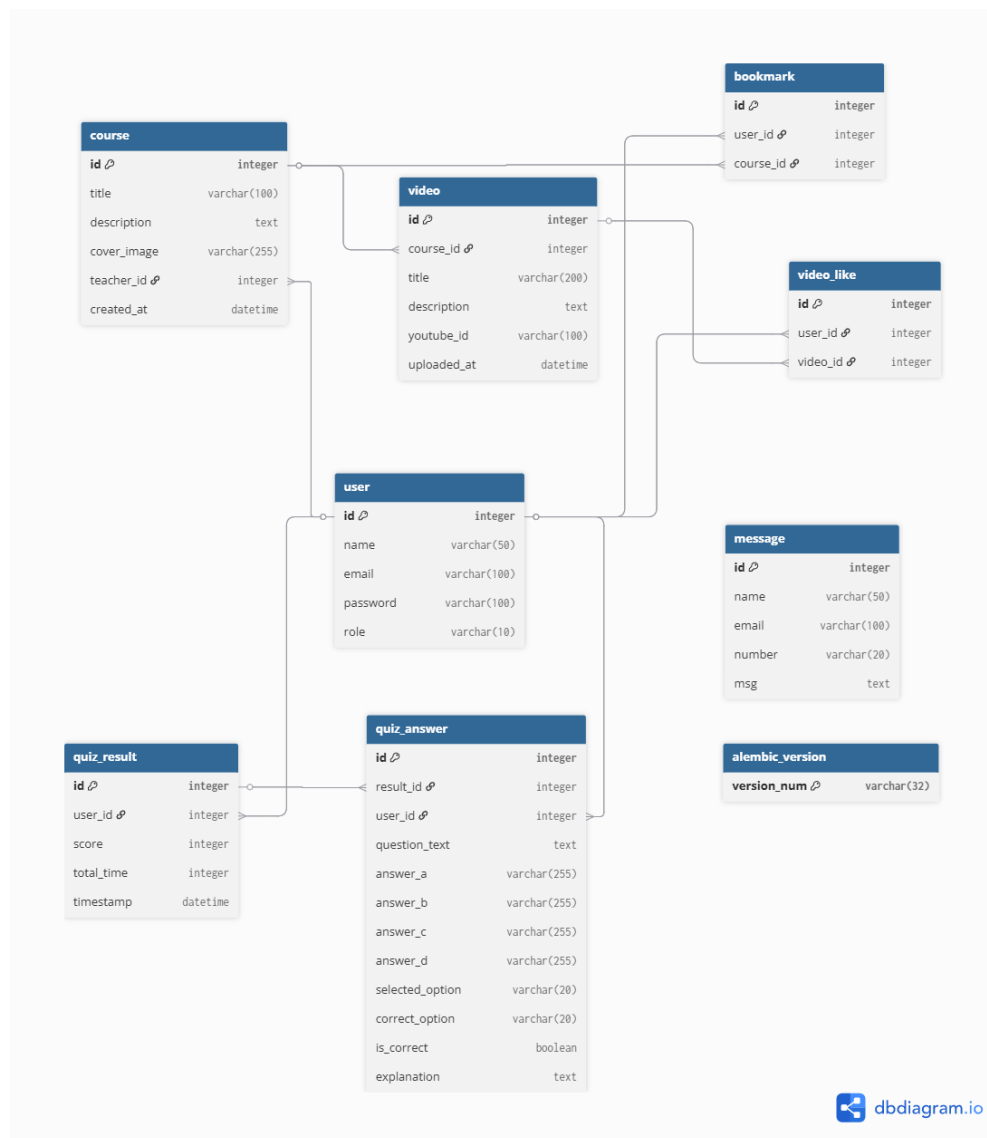


圖 2-15 資料庫之 ERD(Entity-relationship Diagram)

2.5.1 資料表設計

- User (使用者資訊)
 - (1) 儲存使用者的基本資訊，包含姓名、聯絡信箱、密碼及身分。
 - (2) 作為被多個資料表的參照的主要資料表
- Course (課程資訊)：
 - (1) 儲存教師開設的基本資訊，包含標題、課程概述、封面圖片、教師 id 及上傳時間。
- Video (影片資訊)：
 - (1) 包含影片資訊，如影片 id、課程 id、影片標題、影片說明、youtube_id、上傳時間。
 - (2) 透過 Youtube Data API 上傳影片，因此每個影片都會有一個獨立的 youtube_id。
- Bookmark (課程收藏)：
 - (1) 記錄使用者的課程收藏情形，由使用者 id 及課程 id 組成。
- VideoLike (影片喜歡紀錄)：
 - (1) 儲存學生對影片按讚的紀錄，由使用者 id 及影片 id 組成
- QuizAnswer (作答詳情)：
 - (1) 儲存學生在每次測驗每一題的作答情形，包含選項內容、正解、是否答對、解析等。
- QuizerResult (測驗結果)：
 - (1) 紀錄學生每次選擇題測驗的總分、作答時間等。

- Message (聯絡表單)：

(1) 紀錄使用者聯絡我們頁面所提交的意見或問題，欄位包括姓名、信箱、電話與訊息內容。

(2) 設定 Gmail 的 SMTP 伺服器，並指定與郵件伺服器通訊時使用 port 587，代表 TLS(加密傳輸)，並且取得表單內容，處理完再寄送至指定的郵件，並將此次表單內容儲存至 Message 這個資料表中。

- Alembic_Version (內部版本紀錄)：

(1) 由 Flask-Migrate 自動建立，追蹤資料庫版本以管理資料庫狀態。

2.5.2 資料表參照情形

為確保資料表間能建立清楚的資料關聯，本系統的資料表間透過外鍵 (foreign key) 進行連結。以下以 Primary Key(被參照)-> Foreign key(參照)表示。

Primary Key(主鍵)	->Foreign Key(外來鍵)
User.id	Course.teacher_id
	Bookmark.user_id
	VideoLike.user_id
	QuizResult.user_id
	QuizAnswer.user_id
Course.id	Video.course_id
	Bookmark.course_id
Video.id	VideoLike.video_id
QuizResult.id	QuizAnswer.result_id

表 2-1 資料表間的參照關係

2.6 第三方 API

本平台結合多種第三方 API，達成即時批改、延伸學習等目標，顯著減少傳統教學中繁瑣的人工檢閱及回饋，透過這些 API，大幅充實平台的應用。以下為本平台使用到的 API：

- Cohere API：作為一個強大的自然語言模型，可依據輸入的內容 (prompt) 生成回應，在此平台中作為 AI 助教，針對選擇題題目進行解析，以及對錯誤的程式碼提供修正建議及說明，幫助學生即時理解錯誤原因。
- QuizAPI：即時提供隨機題目，豐富平台中的測驗內容。
- Judge0 API：一個線上程式碼編譯平台，支援多種語言如 C++、Python、Java，負責提供程式碼編譯及回傳執行結果。搭配 Cohere API 讓學生能夠即時除錯。
- YouTube Data API：用於管理課程影片的上傳，教師可透過此平台上傳教學影片至 YouTube。
- Google Custom Search API：負責擷取網頁內容，以題目及 Cohere 解析作為根據蒐尋相關教學資源，作為做題後的知識補強及延伸學習。

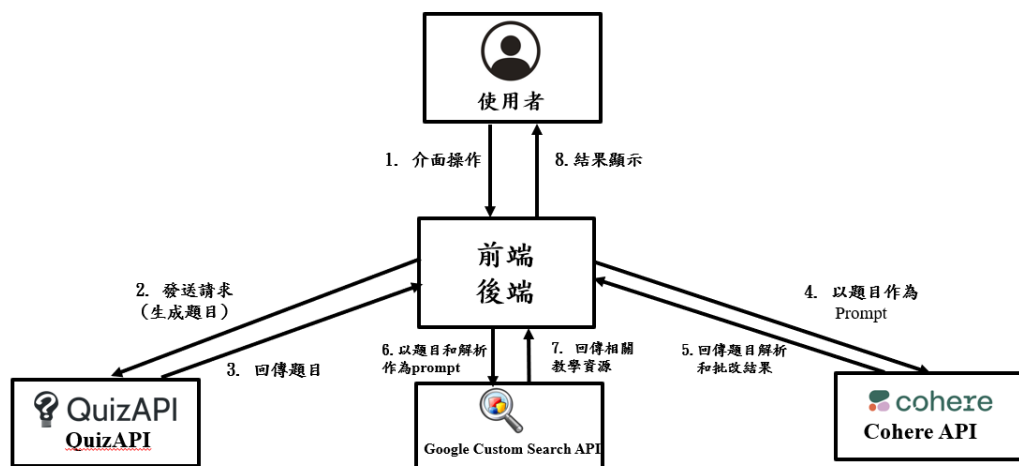


圖 2-16 選題題題目的批改流程

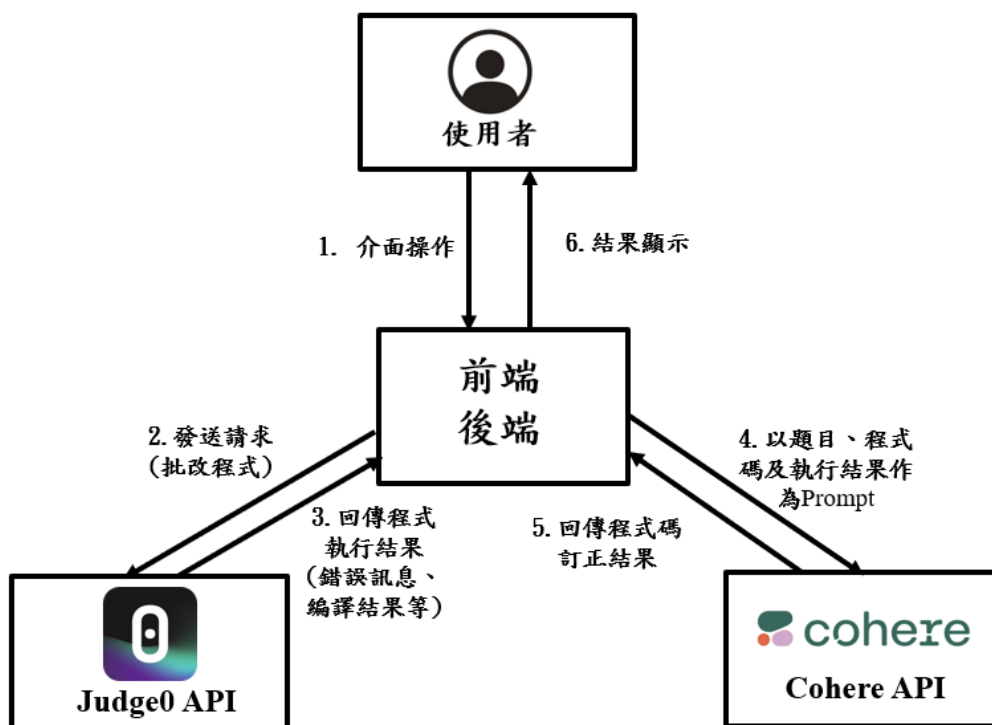


圖 2-17 程式實作的批改流程

2.7 排程與自動化

此功能透過 APScheduler 建立每日排程任務，為每一位使用者產生一份

「測驗分數變化圖表」與「每題作答詳解」的個人化 PDF 報告。此功能幫助學生追蹤自己的學習成果，並且可以回顧曾作答的題目。

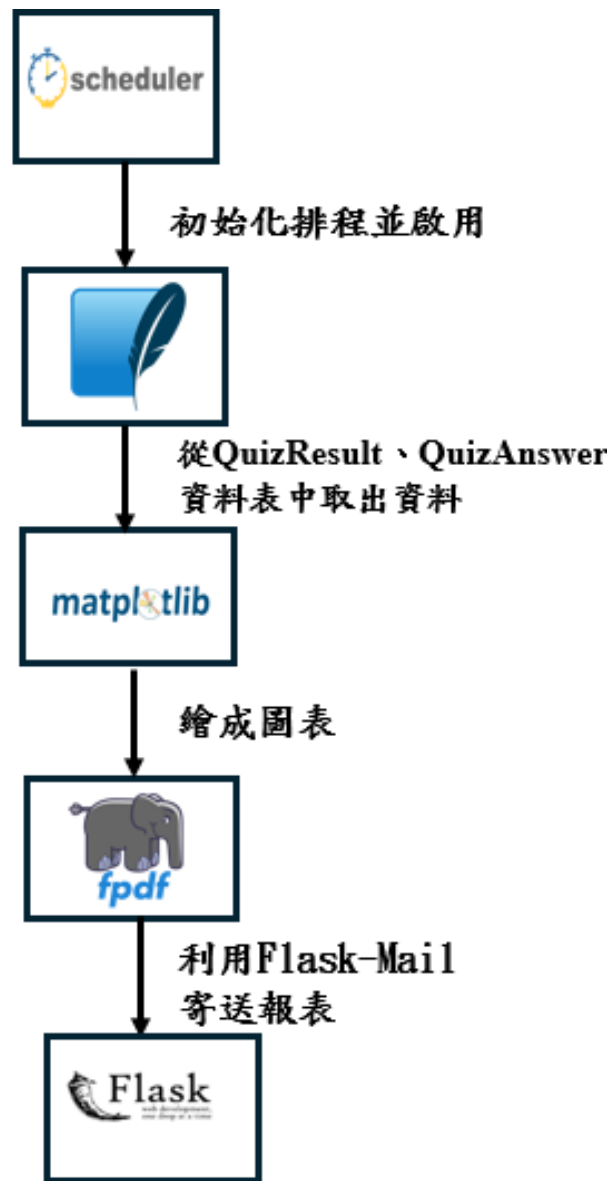


圖 2-17 自動排程流程圖

2.8 爬蟲與資料擷取

透過 Google Custom Search API，自動搜尋與題目相關的學習內容或教學資源，系統會根據題目與解析自動送出查詢，並回傳三筆最相關的教學文章。可

透過此一功能增加學生對於題目的理解。

和傳統的爬蟲不同，並非直接解析 HTML，但同樣具備擷取網頁資訊的功能。

```
def google_search(query, num_results=3):
    url = "https://www.googleapis.com/customsearch/v1"
    params = {
        "key": GOOGLE_API_KEY,
        "cx": GOOGLE_CX,
        "q": query,
        "num": num_results,
    }

    try:
        response = requests.get(url, params=params)
        data = response.json()
        results = []

        for item in data.get("items", []):
            results.append({
                "title": item.get("title"),
                "link": item.get("link"),
                "snippet": item.get("snippet")
            })
        return results
    except Exception as e:
        print("Google 搜尋錯誤:", str(e))
        return []
```

圖 2-18 爬蟲相關程式碼

第三章 結論與未來展望

本平台以「培養學生的運算思維能力與程式設計能力」為核心目標，以 Flask 框架為基礎，實現一個具備互動性、即時性及 AI 輔助的數位學習平台。透過遊戲化的學習方式，融合程式邏輯、問題解決等運算思維元素。同時平台也藉由 Cohere API 即時提供題目與程式碼的解析，並搭配 QuizAPI 與 Judge0 API 完成選擇題的出題與程式方面的批改，整套循序漸進的流程設計讓初學者可以從零開始學起，無負擔的學習。

為了增加平台的可用性，未來可朝以下方向發展：

- 導入響應式網頁設計：使不同裝置的學習者能夠得到相同流暢的使用體驗。
- 增加多種遊戲：增加更多遊戲關卡或是開發不同的遊戲機制，或是用累積積分，讓不同學習者良性競爭，增加學習成效。
- 增加語言種類：針對不同背景的學習者提供不同的語言，拓展使用族群。
- 開發 APP：為提升學習的方便性，未來可進一步將平台開發為應用程式，並支援離線使用，讓學習者能隨時隨地學習。